pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Day Counts and Accrued Interest

WEEK 2 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: what the buyer owes the seller

The Desk Lead's Note

We buy 1,000,000 of Quorvane Energy, settling Monday 30 November, two months into its coupon period. The seller wants paying for the interest it earned: how much per 100 and in dollars, and why does a calendar give another number? Put the day counts and accrued interest in "bondmath", tested against Excel. An assistant has offered to tidy that code; read its change before you take it. Last, show me the holdings file's "accrued" column is right for all 200 bonds.

The Four Bonds

cusip	ticker	coupon_pct	maturity
99001BZA7	QVNE	5.250	2031-09-30
99002CZA4	DNMR	6.125	2033-09-30
99003DZA1	PLWK	4.750	2029-09-30
99004EZA8	HLVF	7.500	2036-09-30

All four accrue from 30 September 2026 and next pay on 31 March 2027, as notebook 06's "coupon_dates" found.

```
...............   [100%]
15 passed in 0.50s
```

The module starts the day as notebook 06 left it: eight functions, 15 tests.

Notebook 06 found the dates; today counts the days between them.

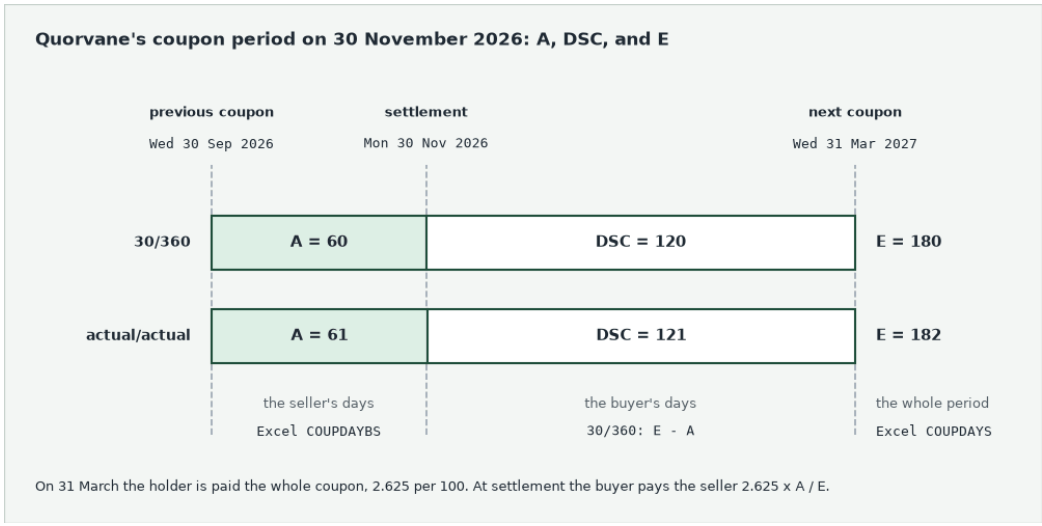
7.1 Why the buyer pays accrued interest

Quorvane pays its whole coupon, 5.25 / 2 = 2.625 per 100, on 31 March to the holder on record. The seller held it from 30 September to 30 November and earned part of that coupon. So at settlement the buyer pays the seller that part, the accrued interest, on top of the quoted price:

accrued = coupon_pct / frequency x A / E

A is the days accrued, from the previous coupon to settlement, and E the days in the coupon period. A day count convention is the bond's rule for counting them.

```
previous_coupon = date(2026, 9, 30)
next_coupon = date(2027, 3, 31)
days_held = (settlement - previous_coupon).days
days_in_period = (next_coupon - previous_coupon).days
accrued_actual = 5.25 / 2 * days_held / days_in_period
print(f'{days_held} of {days_in_period} days: accrued {accrued_actual:.6f} per 100')
61 of 182 days: accrued 0.879808 per 100
```



DSC is the buyer's days, from settlement to the next coupon.

Excel side by side. "COUPDAYBS" is Excel's A and "COUPDAYS" its E; "YEARFRAC(start, end, 0)" is the year fraction on 30/360.

```
Calendar days
=DATE(2026,11,30)-DATE(2026,9,30)
30/360 days
=YEARFRAC(DATE(2026,9,30),DATE(2026,11,30),0)*360
Accrued per 100
=100*5.25/100/2*COUPDAYBS(DATE(2026,11,30),DATE(2031,9,30),2,0)/COUPDAYS(DATE(2026,11,30),DATE(2031,9,30),2,0)
```

7.2 The 30/360 formula, and why it needs rules

30/360 counts with one formula, Y, M, and D being the year, month, and day of the start (1) and the end (2):

Days = 360 x (Y2 - Y1) + 30 x (M2 - M1) + (D2 - D1)

```
end = date(2026, 9, 30)
for start in [date(2026, 8, 16), date(2026, 8, 31)]:
    days = 360 * (end.year - start.year) + 30 * (end.month - start.month) + (end.day - start.day)
    print(f'{start} to {end}: {days} days')
2026-08-16 to 2026-09-30: 44 days
2026-08-31 to 2026-09-30: 29 days
```

44 is right, 30 x 1 + (30 - 16), but 31 August to 30 September is one whole month, and the formula says 29. So US 30/360 first moves D1 or D2 at month ends, by four rules, in order.

Excel side by side

Job	Excel	Result
The formula, start in A1 and end in B1	`=360*(YEAR(B1)-YEAR(A1))+30*(MONTH(B1)-MONTH(A1))+(DAY(B1)-DAY(A1))`	44, then 29
Rule 3 in the sheet	`the same, with MIN(DAY(A1),30) for DAY(A1)`	30

Every rule is about a month end; where a rule can be read two ways, the course counts as Excel does.

7.3 days_30_360, the four rules in order

```
%%writefile -a bondmath.py
def days_30_360(start: date, end: date) -> int:
    d1, d2 = start.day, end.day
    start_is_feb_end = start.month == 2 and start.day == calendar.monthrange(start.year, 2)[1]
    end_is_feb_end = end.month == 2 and end.day == calendar.monthrange(end.year, 2)[1]
    if start_is_feb_end:
        d1 = 30
    if start_is_feb_end and end_is_feb_end:
        d2 = 30
    if start.day == 31:
        d1 = 30
    if end.day == 31 and start.day >= 30:
        d2 = 30
    return 360 * (end.year - start.year) + 30 * (end.month - start.month) + (d2 - d1)
```

"days_30_360(start, end)" first asks whether each date is the last day of February ("calendar.monthrange", as in notebook 06), then applies the four rules in order, then the formula.

Every count equals "COUPDAYBS"; the calendar parts from it only at month ends.

Rule Cases Against Excel

case_id	couppcd	settlement	coupdaybs	days_30_360	calendar
99120QAD1	2026-08-16	2026-09-30	44	44	45
settle_1_mar_after_feb_end	2027-02-28	2027-03-01	1	1	1
mat_aug_31	2026-08-31	2026-09-30	30	30	30
settle_31_oct_mat_30_jul	2026-07-30	2026-10-31	90	90	93
settle_31_oct_mat_15	2026-10-15	2026-10-31	16	16	16
settle_feb_28_nonleap_mat_15	2026-11-15	2027-02-28	103	103	105

The test, and DAYS360, a lookalike

"BASIS" and "bond_inputs" turn a reference row's text into bondmath's units; today's other tests and notebook 08 use them. The rule test is the hand table, each "COUPDAYBS" case checked against its reference row, so it cannot drift from Excel.

```
%%writefile -a test_bondmath.py
BASIS = {"0": "30/360", "1": "ACT/ACT"}  # Excel's basis number to bondmath's name
def bond_inputs(row: dict) -> tuple:
    return (date.fromisoformat(row["settlement"]), date.fromisoformat(row["maturity"]),
            float(row["coupon_pct"]), float(row["yield_pct"]), int(row["frequency"]), BASIS[row["basis"]])
def test_days_30_360_rule_cases():
    cases = [
        (date(2026, 8, 16), date(2026, 9, 30), 44, "COUPDAYBS"),    # no rule applies
        (date(2027, 2, 28), date(2027, 3, 1), 1, "COUPDAYBS"),      # rule 1: last day of February counts as 30
        (date(2027, 2, 28), date(2028, 2, 29), 360, "YEARFRAC"),    # rule 2: both on the last day of February
        (date(2026, 8, 31), date(2026, 9, 30), 30, "COUPDAYBS"),    # rule 3: a 31st start counts as 30
        (date(2026, 7, 30), date(2026, 10, 31), 90, "COUPDAYBS"),   # rule 4: a 31st end after a 30th start
        (date(2026, 10, 15), date(2026, 10, 31), 16, "COUPDAYBS"),  # a 31st end after a 15th start stays 31
        (date(2026, 11, 15), date(2027, 2, 28), 103, "COUPDAYBS"),  # an end on 28 February is not changed
    ]
    for start, end, expected, source in cases:
        assert bondmath.days_30_360(start, end) == expected, (start, end)
        if source == "COUPDAYBS":
            matches = [row for row in REFERENCE if row["basis"] == "0" and row["couppcd"] == start.isoformat()
                       and row["settlement"] == end.isoformat()]
            assert matches, f"no reference row with previous coupon {start} and settlement {end}"
            assert int(matches[0]["coupdaybs"]) == expected
```

```
................   [100%]
16 passed in 0.50s
```

Excel's "DAYS360(start, end)" sounds like the same count, and unless the start is the last day of February it is. It never applies rule 2, and its rule 4 reads D1 after rule 1.

```
2027-02-28 to 2028-02-29 360
2026-02-28 to 2026-10-31 241
2027-02-28 to 2027-02-28 0
```

Job	Result
Rule 2	360; DAYS360 359
Rule 4 after a February start	241; DAYS360 240
The same date twice	0; DAYS360 -2

"days_30_360" counts as "COUPDAYBS" and "YEARFRAC(..., 0)" do; "DAYS360" does not, on February month ends.

7.4 A, E, and DSC: coupon_period_days

```
%%writefile -a bondmath.py
def coupon_period_days(settlement: date, maturity: date, frequency: int = 2,
                       basis: str = "30/360") -> tuple[int, int, int]:
    if basis not in ("30/360", "ACT/ACT"):
        raise ValueError(f'basis must be "30/360" or "ACT/ACT", not {basis!r}')
    dates = coupon_dates(settlement, maturity, frequency)
    previous_coupon, next_coupon = dates[0], dates[1]
    if basis == "30/360":
        days_accrued = days_30_360(previous_coupon, settlement)
        days_in_period = 360 // frequency
        days_to_next = days_in_period - days_accrued
    else:
        days_accrued = (settlement - previous_coupon).days
        days_in_period = (next_coupon - previous_coupon).days
        days_to_next = (next_coupon - settlement).days
    return days_accrued, days_in_period, days_to_next
```

The period drawn in 7.1 has three counts, in Excel's letters:

A, the days accrued: "COUPDAYBS".

E, the days in the period: "COUPDAYS", 360 / frequency on 30/360, so 180 for Quorvane.

DSC, the days from settlement to the next coupon: E - A on 30/360, the count Excel's "PRICE" uses.

Actual/actual (Excel's basis 1, 'ACT/ACT' in "bondmath") counts all three in calendar days.

```
importlib.reload(bondmath)
for basis in ['30/360', 'ACT/ACT']:
    print(basis, bondmath.coupon_period_days(settlement, quorvane_maturity, 2, basis))
quorvane = week2.loc['99001BZA7']
print('Excel, basis 0:', quorvane['coupdaybs'], quorvane['coupdays'], quorvane['coupdaysnc'])
30/360 (60, 180, 120)
ACT/ACT (61, 182, 121)
Excel, basis 0: 60 180 120
```

A + DSC = E on both bases, and Excel's three functions give the 30/360 line.

E on actual/actual, and where COUPDAYSNC is not E - A

```
basis1 = reference[reference['basis'] == 1]
period_days = (pd.to_datetime(basis1['coupncd']) - pd.to_datetime(basis1['couppcd'])).dt.days
print(len(basis1), 'basis 1 rows;', (basis1['coupdays'] != period_days).sum(),
      'where COUPDAYS differs')
28 basis 1 rows; 12 where COUPDAYS differs
```

Not always, so on actual/actual E is "COUPNCD - COUPPCD", the length "PRICE" uses.

```
basis0 = reference[reference['basis'] == 0]
nc_differs = basis0['coupdaysnc'] != basis0['coupdays'] - basis0['coupdaybs']
print(len(basis0), 'basis 0 rows;', nc_differs.sum(), 'where COUPDAYSNC is not E - A')
278 basis 0 rows; 7 where COUPDAYSNC is not E - A
```

Every one has a last day of February as settlement, previous coupon, or next coupon. "99031FAF8" matures on 28 February 2036, not a month end in a leap year, so its next coupon is 28 February 2027. E - A is 148 days, "COUPDAYSNC" 150, and the calendar 151; "PRICE" uses E - A, so "bondmath" does too.

```
%%writefile -a test_bondmath.py
def test_coupon_period_days_match_excel():
    for row in REFERENCE:
        settlement, maturity, _, _, frequency, basis = bond_inputs(row)
        days_accrued, days_in_period, days_to_next = bondmath.coupon_period_days(settlement, maturity, frequency, basis)
        assert days_accrued == int(row["coupdaybs"]), row["case_id"]
        if basis == "30/360":
            assert days_in_period == int(row["coupdays"]), row["case_id"]
            assert days_to_next == int(row["coupdays"]) - int(row["coupdaybs"]), row["case_id"]
        else:
            period_start, period_end = date.fromisoformat(row["couppcd"]), date.fromisoformat(row["coupncd"])
            assert days_in_period == (period_end - period_start).days, row["case_id"]
            assert days_to_next == int(row["coupdaysnc"]), row["case_id"]
```

```
.................   [100%]
17 passed in 0.50s
```

Job	Result
A	60
E	180
DSC	120; 180
A on actual/actual	153 (30/360: 150)
E on actual/actual	182; COUPDAYS 181
99031FAF8 DSC	148; COUPDAYSNC 150

7.5 accrued_interest: one coupon times A / E

"accrued_interest" is 7.1's formula, with A and E from "coupon_period_days". Unpacking the tuple, "_" sets aside DSC, which accrued does not need.

```
%%writefile -a bondmath.py
def accrued_interest(settlement: date, maturity: date, coupon_pct: float, frequency: int = 2,
                     basis: str = "30/360") -> float:
    days_accrued, days_in_period, _ = coupon_period_days(settlement, maturity, frequency, basis)
    return coupon_pct / frequency * days_accrued / days_in_period
```

```
%%writefile -a test_bondmath.py
def test_accrued_matches_excel():
    for row in REFERENCE:
        settlement, maturity, coupon_pct, _, frequency, basis = bond_inputs(row)
        result = bondmath.accrued_interest(settlement, maturity, coupon_pct, frequency, basis)
        assert result == pytest.approx(float(row["accrued"]), abs=1e-9), row["case_id"]
```

Against Excel, 30 November

ticker	python	excel
DNMR	1.020833	1.020833
PLWK	0.791667	0.791667
HLVF	1.250000	1.250000

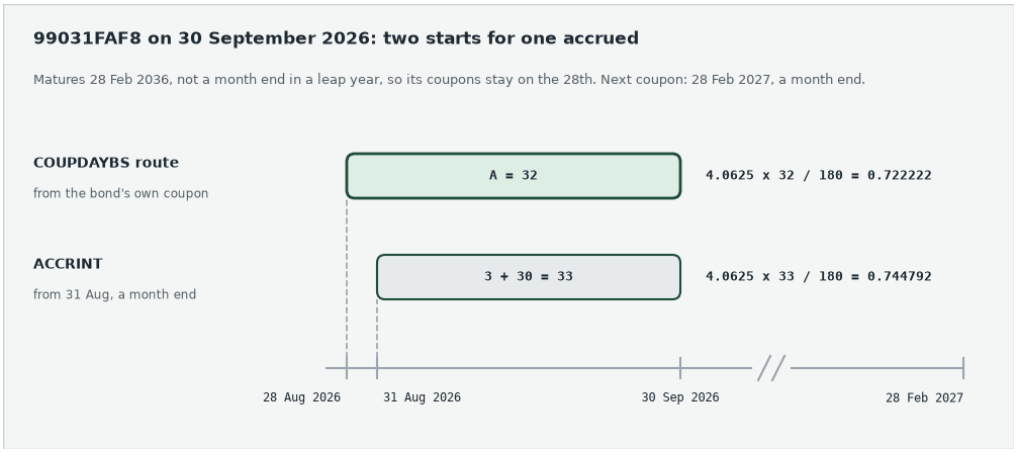
```
..................   [100%]
18 passed in 0.50s
```

ACCRINT, the second route, and where it parts

Excel's "ACCRINT(issue, first_interest, settlement, rate, par, frequency, basis)" also returns accrued interest.

```
blank = reference['accrint'].isna()
differs = (reference['accrint'] - reference['accrued']).abs() > 1e-9
ACCRINT blank (#NUM! in Excel): 21  different: 9
99031FAF8, formula and ACCRINT: [0.7222222222222222, 0.7447916666666666]
```

"ACCRINT" returns "#NUM!" when settlement is a coupon date or the coupon is 0, where the formula gives 0. It counts its own period back from "first_interest", the next coupon, and when that is a month end and maturity is not, it starts on another day. The course follows the "COUPDAYBS" formula, the accrued that "PRICE" takes off in notebook 08.



Job	Result
Accrued, actual/actual	0.879808
Where the routes part	0.722222; ACCRINT 0.744792

7.6 Read a suggested change word by word

```
!git -C "{work}" diff --stat
subject = 'Notebook 07: days_30_360, coupon_period_days, and accrued_interest'
body = 'The first money number of week 2; the counts follow COUPDAYBS and PRICE.'
!git -C "{work}" add bondmath.py test_bondmath.py
!git -C "{work}" commit -q -m "{subject}" -m "{body}"
 bondmath.py   | 75 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 test_bondmath.py | 56 ++++++++++++++++++++++++++++++++++++++++
 2 files changed, 131 insertions(+)
```

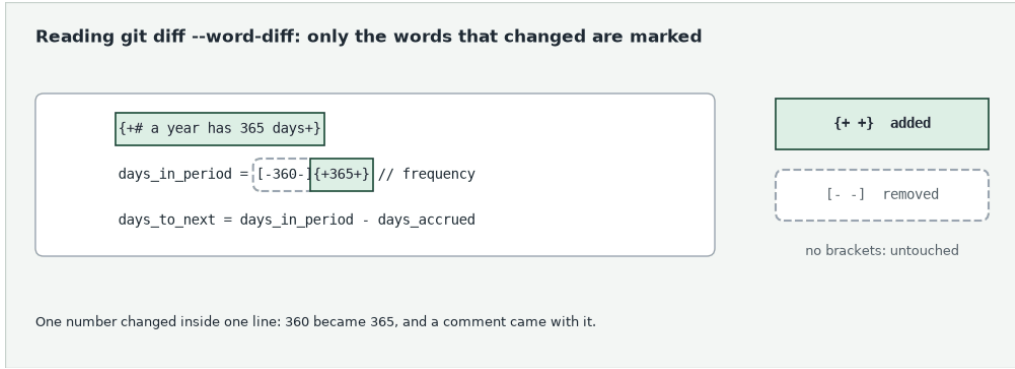
You paste "coupon_period_days" into an AI assistant and ask it to tidy the code. It sends back one changed line, with a comment above it: "a year has 365 days".

```
with open('bondmath.py', encoding='utf-8') as file:
    text = file.read()
new_lines = '# a year has 365 days\n        days_in_period = 365 // frequency'
with open('bondmath.py', 'w', encoding='utf-8') as file:
    file.write(text.replace('days_in_period = 360 // frequency', new_lines))
```

Throw It Away

```
!git -C "{work}" restore bondmath.py
importlib.reload(bondmath)
print(f'Accrued again: {bondmath.accrued_interest(settlement, quorvane_maturity, 5.25):.6f}')
```

```
!git -C "{work}" diff --word-diff bondmath.py
@@ -251,7 +251,8 @@ def coupon_period_days(settlement: date, maturity: date, frequency: int = 2,
   if basis == "30/360":
   # 30/360: A by the 30/360 rules, a fixed period length, and DSC as what is left of it
   days_accrued = days_30_360(previous_coupon, settlement)
   {+# a year has 365 days+}
   days_in_period = [-360-]{+365+} // frequency
```



One number in one line, and a comment. The comment is true of the calendar and beside the point: Quorvane is 30/360, and its period is 180 days.

Read every suggested change word by word and let the tests from Excel judge it: 360 is the convention, not a typo.

7.7 The holdings file's accrued column

"holdings.csv" gives each of its 200 bonds an "accrued" per 100 on 30 September 2026, rounded to 3 decimals. Rounding to 3 decimals moves a number by at most 0.0005, half of the last decimal.

Recompute All 200

```
holdings = pd.read_csv('holdings.csv')
holdings['accrued_python'] = [
    bondmath.accrued_interest(date.fromisoformat(a), date.fromisoformat(m), c)
    for a, m, c in zip(holdings['as_of_date'], holdings['maturity'], holdings['coupon'])]
holdings['gap'] = (holdings['accrued_python'] - holdings['accrued']).abs()
print(f"Largest gap: {holdings['gap'].max():.10f} per 100")
```

Check Against the Limit

```
assert holdings['gap'].max() <= 0.0005 + 1e-9, 'a bond is off by more than rounding'
```

"99031FAF8" is in the file at 0.722: the "COUPDAYBS" route, not ACCRINT's 0.7448.

Bring it home, and what you can now do

Copy the work folder's three files into your own "my_bondmath" folder, then test and commit there. The copy replaces your files, so anything of your own goes into the work folder first. In a PowerShell terminal in VS Code, with the course's ".venv" active, use the work folder name your setup cell printed:

```
Set-Location "$HOME\projects\my_bondmath"
$work = "$env:TEMP\pfi_course2_07_k4w7q2xr"
Copy-Item "$work\bondmath.py", "$work\test_bondmath.py", "$work\.gitignore" .
python -m pytest -q
git diff --stat
git add bondmath.py test_bondmath.py .gitignore
git commit -m "Notebook 07: days_30_360, coupon_period_days, and accrued_interest"
```

Keep the habit: read any suggested edit with "git diff --word-diff", run "python -m pytest -q", and "git restore" it if a test fails. In Colab, download the three files instead. The work folder stays in your temp folder until you delete it.

You Can Now

Compute accrued interest as one coupon times A / E, and say why the buyer pays it.

Count days on US 30/360 with "days_30_360" or "YEARFRAC(..., 0)", and say where "DAYS360" differs.

Count a period on actual/actual, with E as "COUPNCD - COUPPCD".

Return A, E, and DSC with "coupon_period_days", tied to Excel's "COUP" functions.

Compute accrued with "accrued_interest", and say when "ACCRINT" differs.

Read a suggested change with "git diff --word-diff", judge it with pytest, and discard it with "git restore".

Predict first: 30/360, and a February start

Predict 1

30/360 counts every month as 30 days: two months are 60 days, and a half-year 180. Against the calendar's 0.879808, the accrued on 30/360 is:

A) the same

B) a little higher

C) a little lower

```
accrued_30 = 5.25 / 2 * 60 / 180
print(f'30/360: accrued {accrued_30:.6f} per 100')
```

Predict 2

28 February 2027 is the last day of February, and 31 March is a 31st: a month end at each side. What does Excel count from 28 February to 31 March 2027 on 30/360?

A) 30, one whole month, as 31 August to 30 September should be

B) 31, one month and a day

C) 33, the formula with no rule

```
row = reference.loc['edge_settle_31_mar_after_feb_end']
print('previous coupon', row['couppcd'], ' settlement', row['settlement'],
      ' COUPDAYBS', row['coupdaybs'])
```

Predict first: 365 days, and the tests

Predict 3

Quorvane's A stays 60, and the new line makes E "365 // 2". What is Quorvane's accrued per 100 now? Write down a number.

```
importlib.reload(bondmath)
changed = bondmath.accrued_interest(settlement, quorvane_maturity, 5.25)
print(f'Accrued with the change: {changed:.6f}')
```

Predict 4

The change moves E from 180 to 182 and nothing else. Of the 18 tests:

A) none fail, the code still runs

B) one, the accrued test

C) two, the period test and the accrued test

```
import subprocess
import sys
command = [sys.executable, '-m', 'pytest', '-q', '-p', 'no:cacheprovider', '--tb=no']
result = subprocess.run(command, cwd=work, capture_output=True, text=True)
print(result.stdout + result.stderr)
```

Predict first: the largest gap to the file

Predict 5

Across the 200 bonds, how close does the largest gap between "accrued_interest" and the file come to that limit?

A) on the limit, 0.0005000000

B) just inside, about 0.00049

C) well inside, about 0.0004

```
holdings = pd.read_csv('holdings.csv')
holdings['accrued_python'] = [
    bondmath.accrued_interest(date.fromisoformat(a), date.fromisoformat(m), c)
    for a, m, c in zip(holdings['as_of_date'], holdings['maturity'], holdings['coupon'])]
holdings['gap'] = (holdings['accrued_python'] - holdings['accrued']).abs()
print(f"Largest gap: {holdings['gap'].max():.10f} per 100")
```

The note, answered

The note asked	Answer
Accrued per 100	0.875, 60 of 180 days on 30/360
In dollars	8,750.00 on 1,000,000 of face
Why the calendar differs	61 of 182 days is actual/actual, 0.879808, another convention
Day counts in bondmath	days_30_360, coupon_period_days, accrued_interest, 18 tests over 306 rows
The assistant's change	Rejected: 360 became 365, two tests failed, git restore
The holdings column	200 of 200 within rounding, the largest gap a rounding tie

Be proud: you can say to the cent what a buyer owes a seller on any day, and why two systems may disagree.

Next, Clean and Dirty Price (notebook 08) prices this trade between coupons: the quote plus today's accrued.